

AI 2002

Artificial Intelligence

Lecture 7

Mahzaib Younas

Lecturer, Department of Computer Science

FAST NUCES CFD

Beyond Classical Search

- We have addressed a single category of problems: **observable, deterministic, known environments**
 - where the solution is a sequence of actions
- The search algorithms that we have seen so far are designed to explore search spaces **systematically**.
 - When a **goal** is found, the ***path*** to that goal also constitutes a ***solution*** to the problem.

Beyond Classical Search

- In Many problems, however, the path of the goal is irrelevant.
- Example:
- 8-Queens Problem
 - What matter, is the final configuration of queens, not the order.
 - The factory-floor layout problem
 - The vehicle routing problem

Local Searches

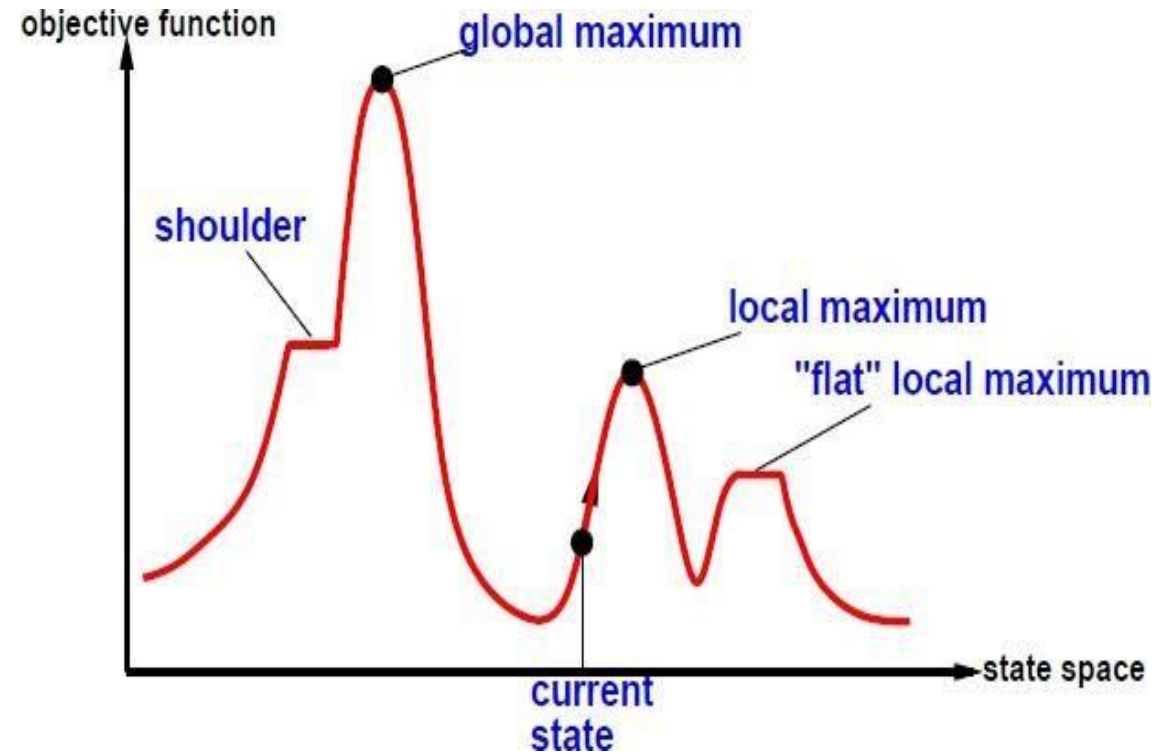
- Operate using a single current node and generally move only to neighbors of that node.
 - No concern with paths followed by the search
 - They are not systematic
- Local Search algorithms have two key advantages
 - They use very little memory
 - They can often find reasonable solutions in large or infinite(continuous) state spaces.

Local Search Algorithms

- The local search algorithms are useful for solving pure optimization problems.
- Optimization problem
 - The aim is to find the best state according to an objective function.
 - The standard search methods do not fit well with optimization problems.

Local Search Algorithms

- State space landscape
- A landscape has both
- Location
 - Defined by the state
- Elevation
 - Defined by the value of the heuristics cost function or objective function.

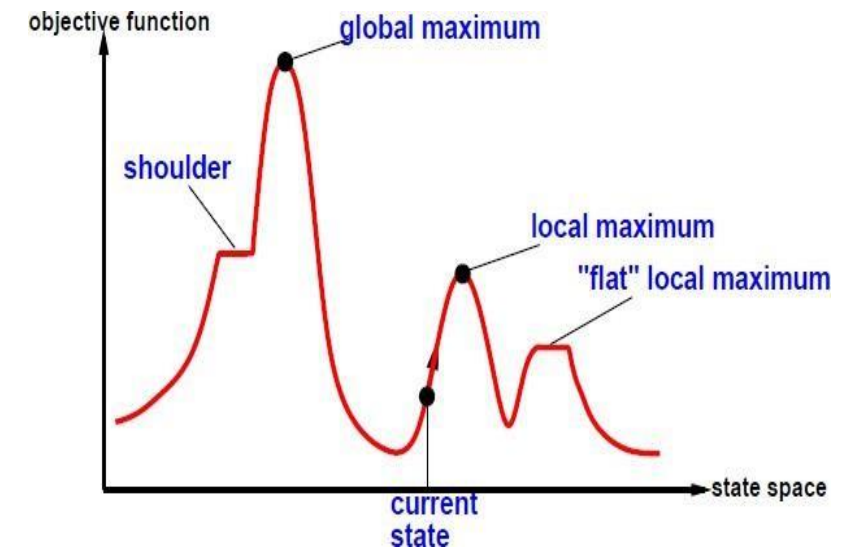


Local Search Algorithm

- A local search problem consists of:
- A state
 - a complete assignment to all of the variables.
- A neighbor relation
 - which states do I explore next?
- A cost function
 - how good is each state?

Basic Terminology

<u>Terminology</u>	<u>Purpose</u>
Shoulder	It is a region having an edge upwards and it is also considered as one of the problems in hill climbing algorithms.
Global Maximum	A state that maximizes the objective function over the entire landscape.
Local Maximum	It is the state which is slightly better than the neighbor states but it is always lower than the highest state.
Flat Maximum	If the neighbor states all having same value, they can be represented by a flat space (as seen from the diagram) which are known as flat local maximums.
Current state	It is the state which contains the presence of an active agent.



State Space Landscape

Global Minimum

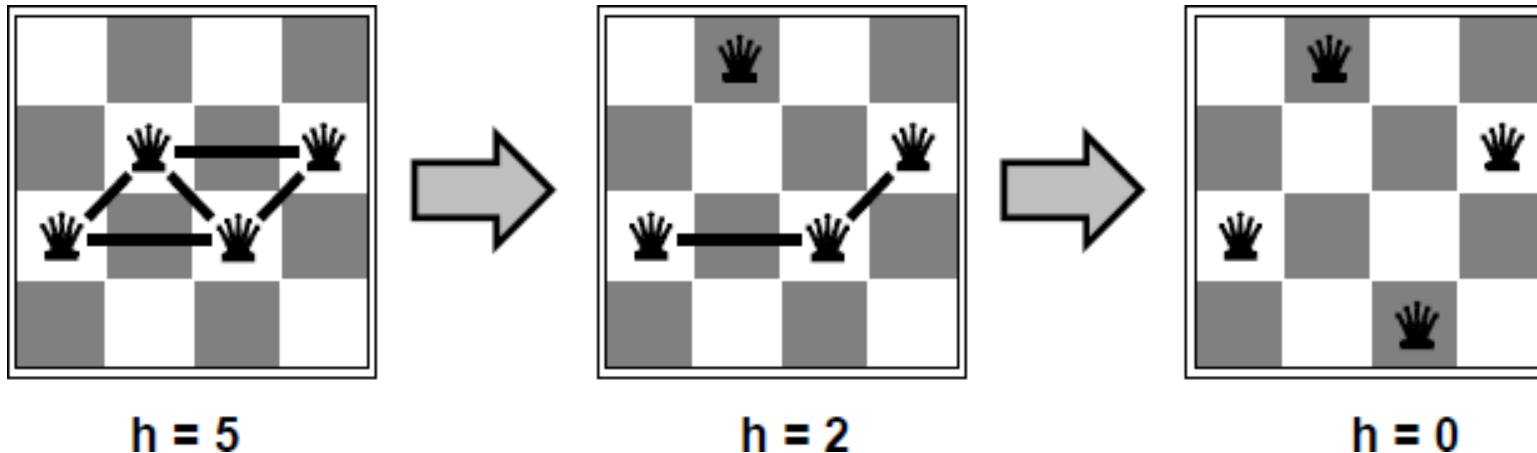
If the elevation corresponds to Cost, then the aim is to find the lowest valley called global minimum.

Global Maximum

If the elevation corresponds to an objective function, then the aim is to find the lowest valley called global maximum.

Example: n-queens Problems

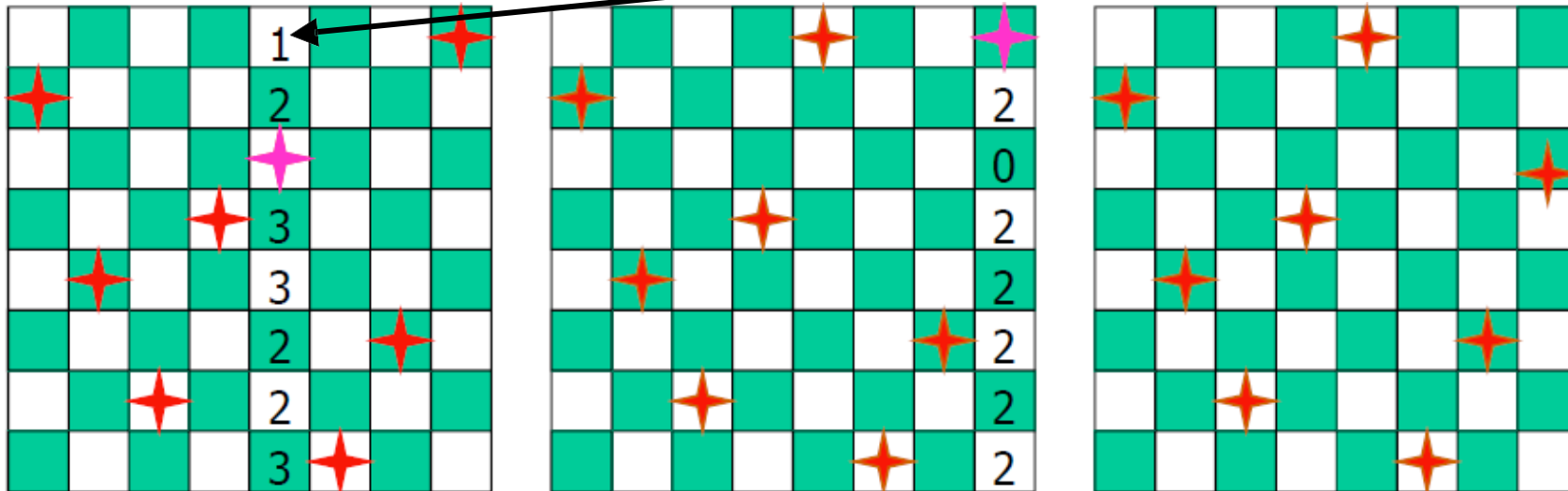
- Put n queens on an $n \times n$ board with no two queens on the same row, column, or diagonal.
- Move a queen to reduce number of conflicts.
- Heuristics h : number of attacks



Example: n-queens Problems

- Put n queens on an $n \times n$ board with no two queens on the same row, column, or diagonal.
- Move a queen to reduce number of conflicts.
- Heuristics h : number of attacks

If queen is placed here, there is only one attack then,



Types of Local Search Algorithm

1. Hill climbing Search
2. Simulated Annealing
3. Local Beam Search

Hill Climbing Search

- It is simply a **loop** that continuously moves in the direction of increasing value—that is, uphill.
- It terminates when it reaches a **“peak”** where no neighbor has a higher value.
 - *does not maintain a search tree*
 - need only
 - *record the state and*
 - *the value of the objective function.*
- Hill climbing only looks towards the **immediate neighbors of the current state.**

Hill Climb Search

function HILL-CLIMBING(*problem*) **returns** a state that is a local maximum

current $\leftarrow$ MAKE-NODE(*problem*.INITIAL-STATE)

loop do

neighbor $\leftarrow$ a highest-valued successor of *current*

if *neighbor*.VALUE $\leq$ *current*.VALUE **then return** *current*.STATE

current $\leftarrow$ *neighbor*

- At each step the current node is replaced by the best neighbor; the neighbor with the highest VALUE,
- If a heuristic cost estimate ***h*** is used, we would find the neighbor with the lowest ***h***.

Hill Climbing Search

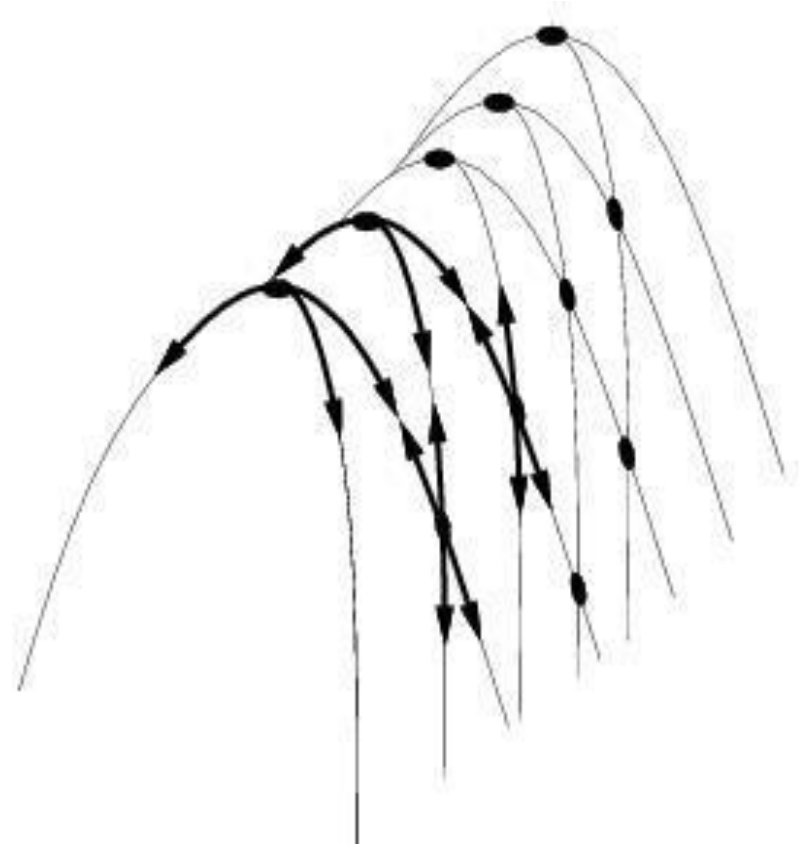
The hill climbing **often gets stuck** for the following reasons:

- **Local maxima:**

- A local maximum is a peak that is higher than each of its neighboring states but lower than the global maximum.

Ridges

- A sequence of local maxima
- Its difficult for greedy algorithms to navigate



Hill Climbing Search

Plateaux

An area of the state space
where evaluation function is
flat.

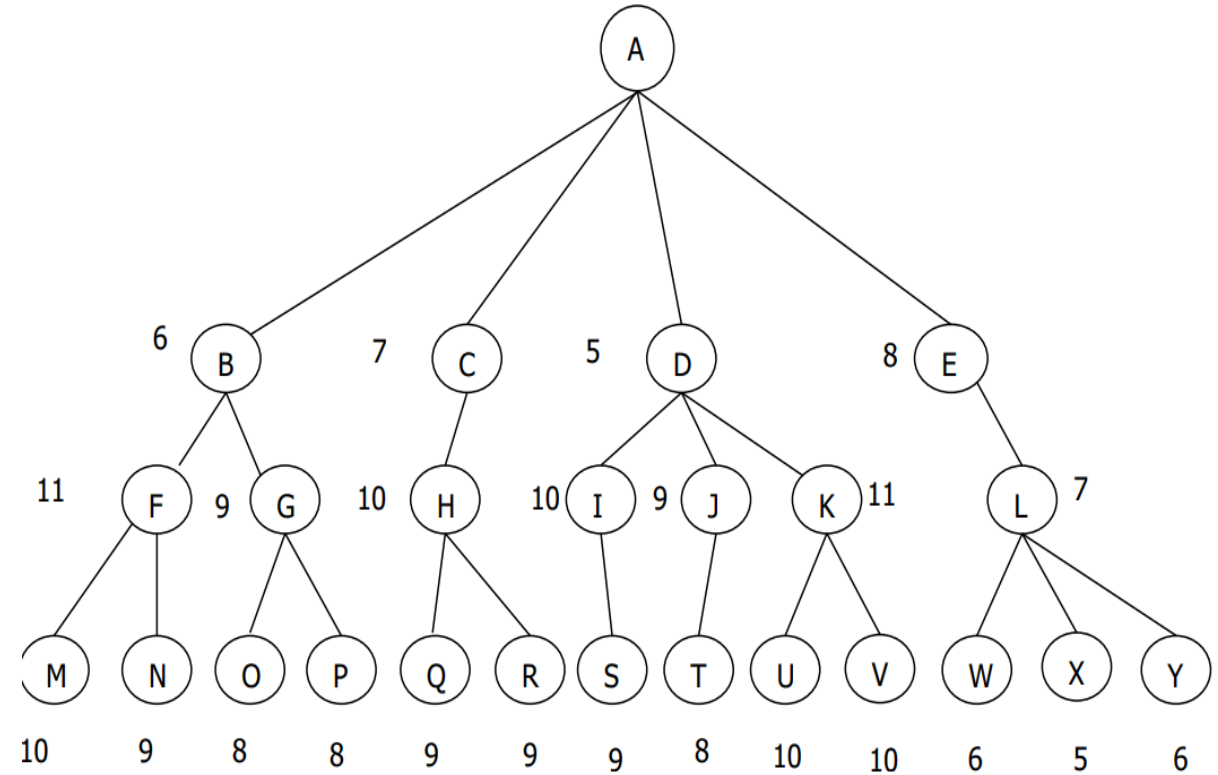
It can be a flat local maximum,
from which no uphill exit

Shoulder

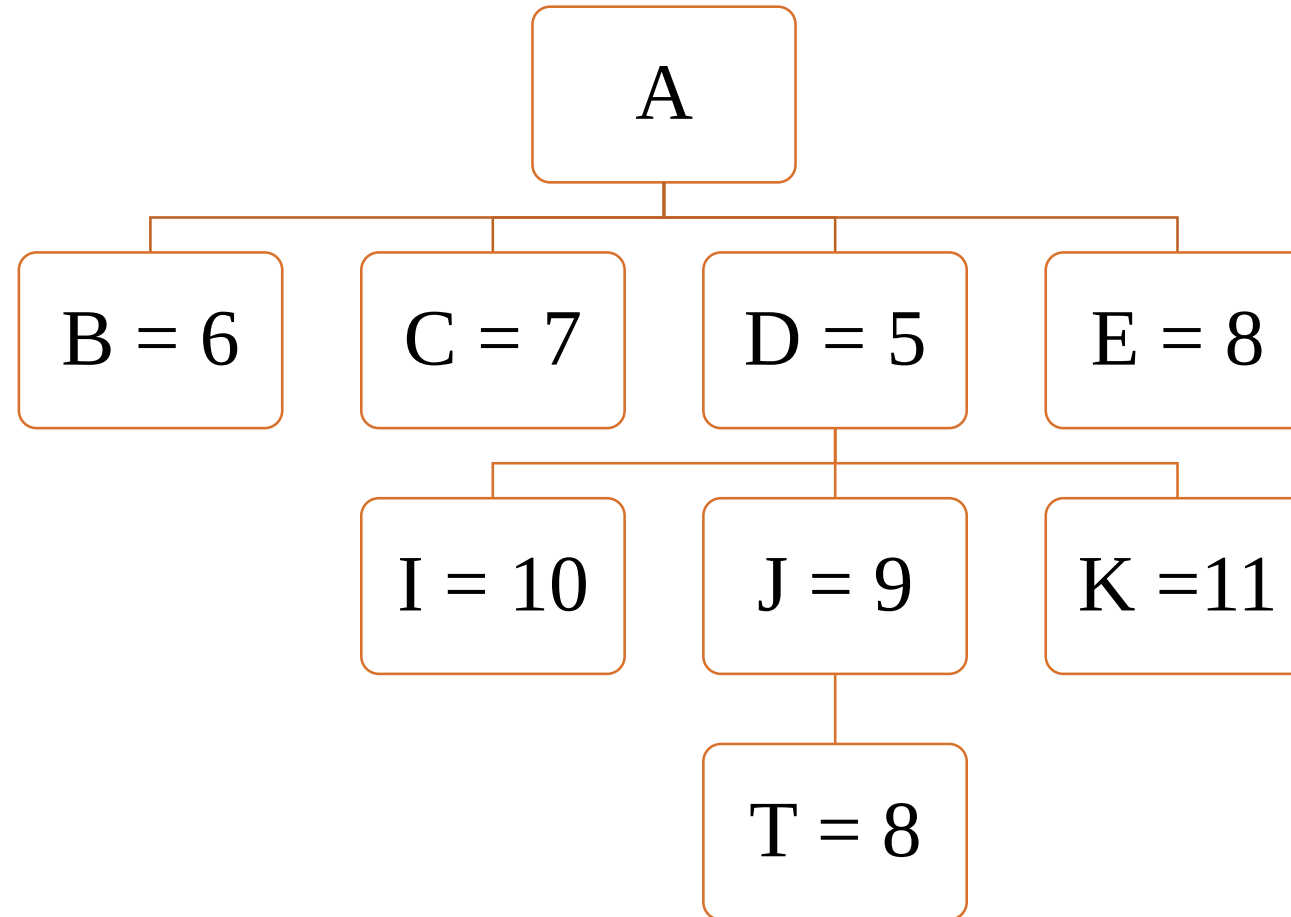
From which progress is
possible

Hill Climb Example

Draw the search tree and write the order of expansion of the nodes when applying: n Hill climbing search.

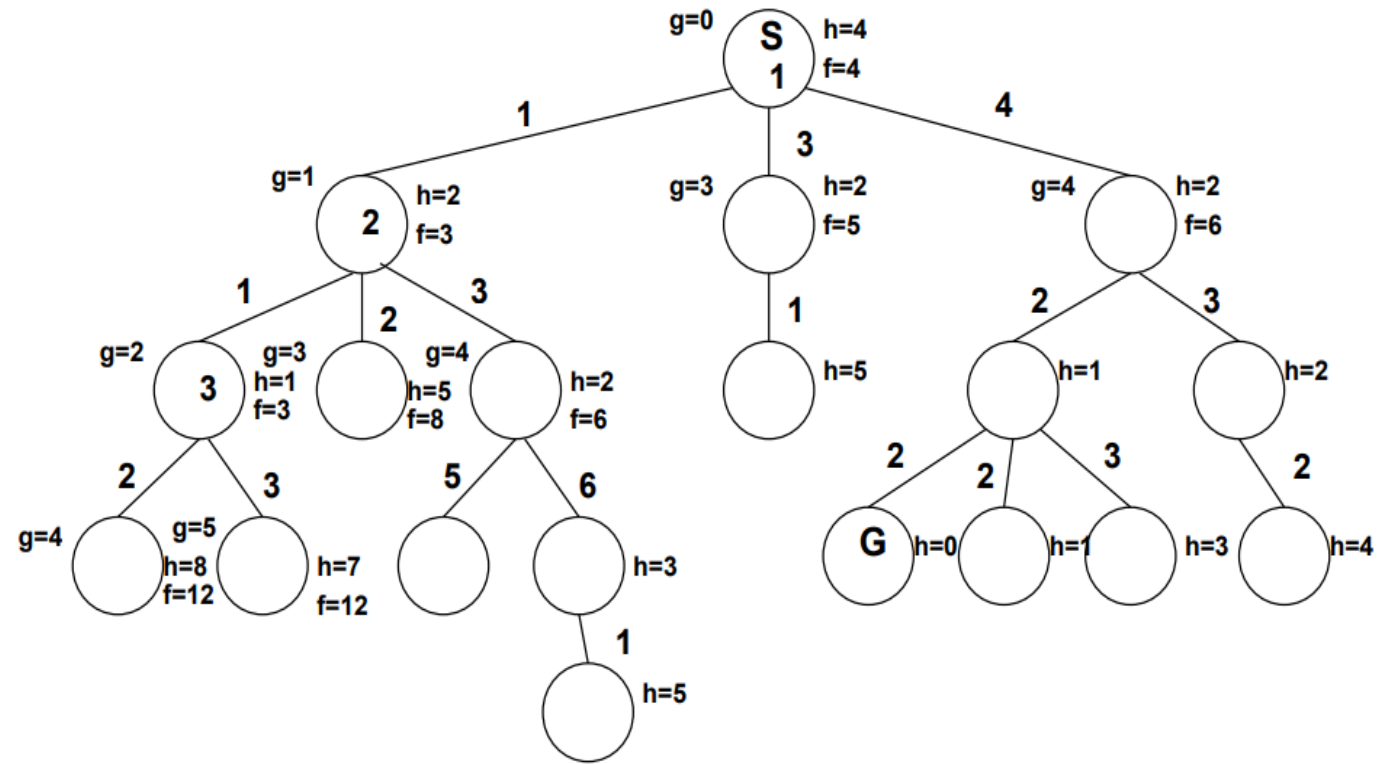


Solution:



Example 2:

- Use the path cost and check the path from S to G.



Example:

- You are an academic coordinator tasked with assigning classrooms for a set of courses $\{c_1, c_2, \dots, c_n\}$, each requiring a specific number of hours $h(c_i) > 0$ per week. You have a set of classrooms, each with a maximum weekly availability of H hours. A classroom can be used for multiple courses as long as the total hours do not exceed H . Your objective is to use the fewest number of classrooms while accommodating all courses.

Answer the following Question:

1. What will be the state for search.
2. What will be the neighbor of state.
3. What will be the evaluation function.
4. What will be the local minimum case.
5. What will be the global minimum case.

Solution:

- **State Representation:**

- Each state represents a specific allocation of **courses to classrooms.**
- For example, a state can be represented as a list of classrooms with their assigned courses and the total hours utilized.

- **Neighbor of State**

- Neighboring states are **generated by small changes to the current state.** This could involve moving a course from one classroom to another or swapping courses between two classrooms, **provided the total hours in any classroom do not exceed H.**

Solution:

- **Evaluation Function:**

- The evaluation function assesses the quality of a state. In this case, the objective is to minimize the number of classrooms used while ensuring no classroom exceeds its weekly availability.
- The evaluation function could be the total number of classrooms with assigned courses, aiming to reduce this number.

- **Local Minimum Case:**

- A local minimum occurs when no neighboring state has a better (lower) evaluation function value than the current state. In other words, it's a state where reallocating or swapping courses does not reduce the number of classrooms used.

Solution:

- **Global Minimum Case**

- The global minimum is the optimal solution where the fewest number of classrooms is used to accommodate all courses. This means that no other possible allocation of courses to classrooms can result in fewer classrooms used while adhering to the constraints.

Types of Hill Climb

- **Stochastic hill-climbing:**
 - Chooses randomly among potential successors
 - Sometimes better than steepest ascent
- **First-choice hill-climbing:**
 - Generates successors randomly and picks first
 - Good for many successors
- **Random restart hill-climbing:**
 - Restarts from randomly generated initial state when failed
 - Roughly 7 iterations with 8-queens problem

Local Beam Search

- Keep track of k states rather than just one
- Start with k randomly generated states
- At each iteration, all the successors of all k states are generated
- If any one is a goal state, stop;
- else select the k best successors from the complete list and repeat.

Local Beam Search

```
function BEAM-SEARCH(problem, k) returns a solution state
  start with k randomly generated states
  loop
    generate all successors of all k states
    if any of them is a solution then return it
    else select the k best successors
```

A local beam search with k states might seem to be nothing more than running **k random restarts in parallel** instead of in sequence.

Local Beam Search

- Local beam search with $k = 1$

We would randomly generate **1 start state**

At each step we would generate all the successors, and retain the **1 best state** Equivalent to **HILL-CLIMBING**

Local Beam Search

- Local beam search with $k = \infty$
- **1 initial state** and no limit of the number of states retained
- We start at initial state and **generate ALL successor**
 - states (no limit how many)
 - If one of those is a goal, we stop
 - Otherwise, we generate all successors of those states (2 steps from the initial state), and continue
- Equivalent to **BREADTH-FIRST SEARCH** except that
 - *each layer is generated all at once.*

Hill Climb vs Local Beam Search

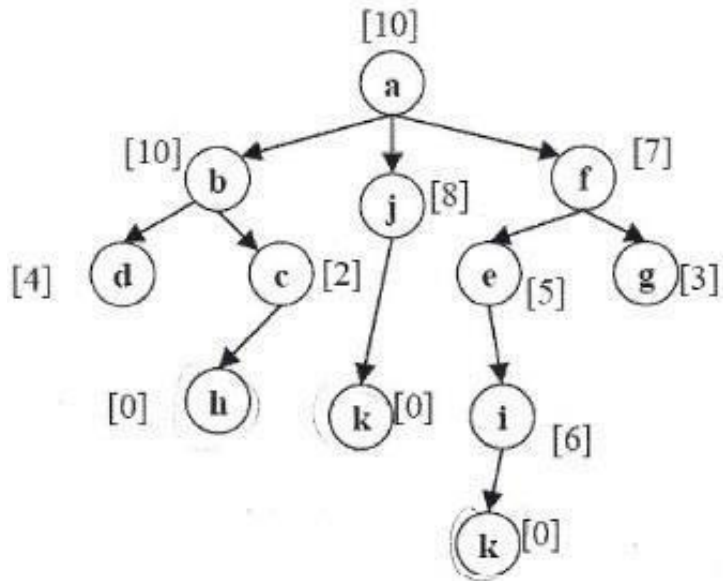
- Hill climbing just explores all nodes in one branch until goal found or not being able to explore more nodes.
- Beam search explores more than one path together. A factor **k** is used to
- determine the number of branches explored at a time.
- If k=2, then two branches are explored at a time. For k=4, four branches are explored simultaneously.
- The branches selected are the **best branches** based on the used
- **heuristic evaluation function.**

Local Beam Search K=2

Current

a

Children

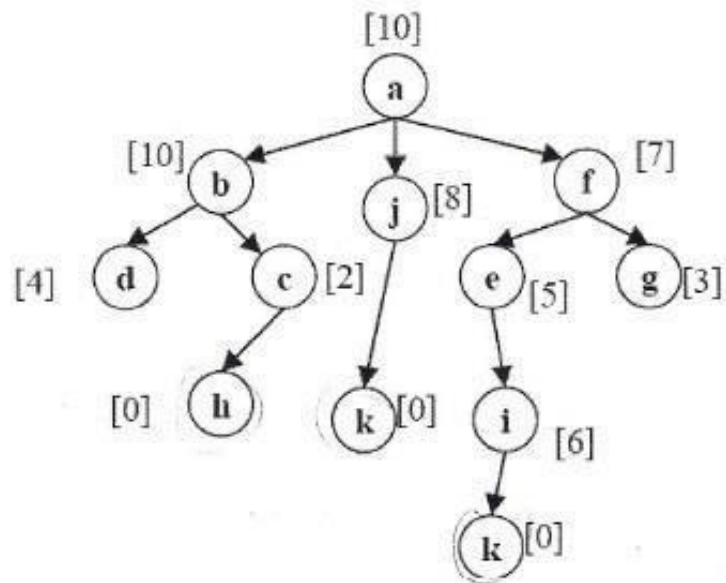


Local Beam Search K=2

Current

a

Children



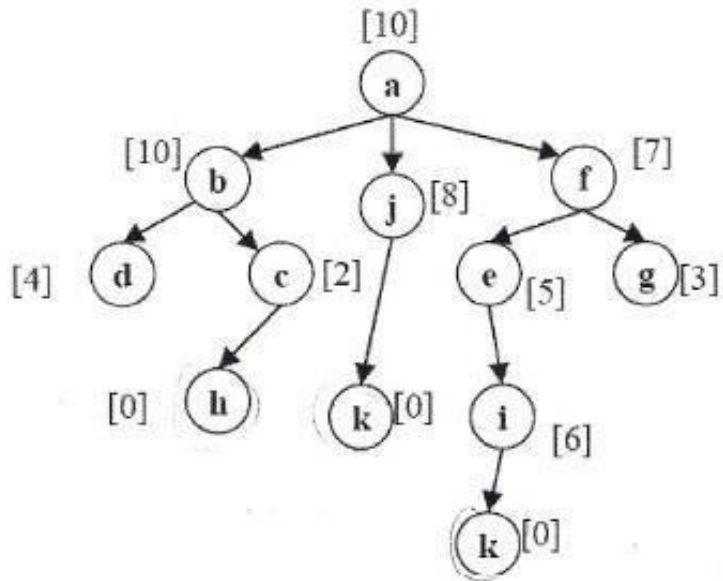
Local Beam Search K=2

Current

a

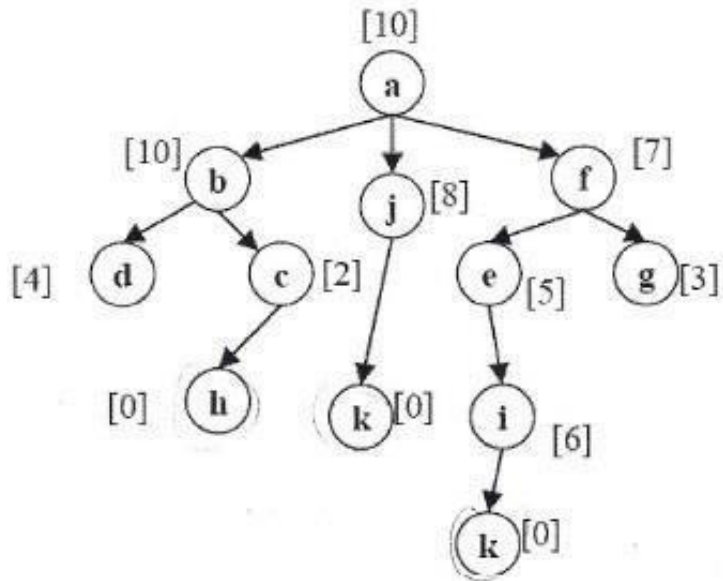
a

Children



Local Beam Search K=2

Current	Children
a	---
a	<i>f7, j8, b10</i>



Best k
Successors

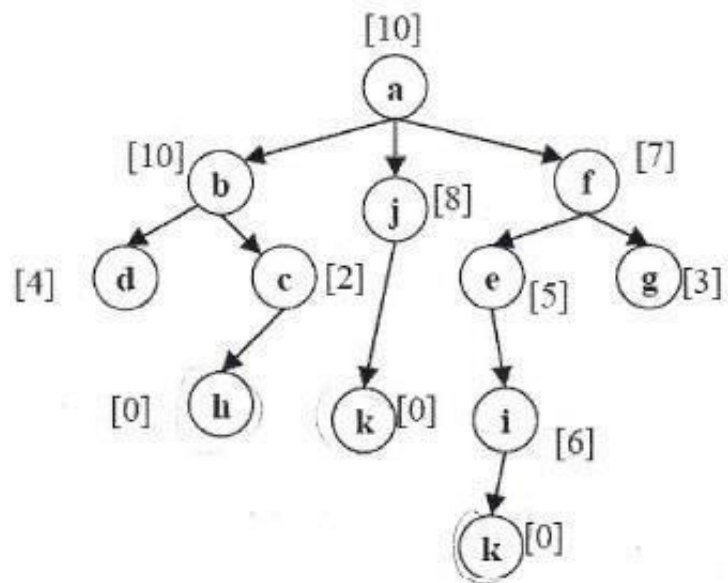
Current

a

a

Children

f7, j8, b10



Local Beam Search K=2

Best k
Successors

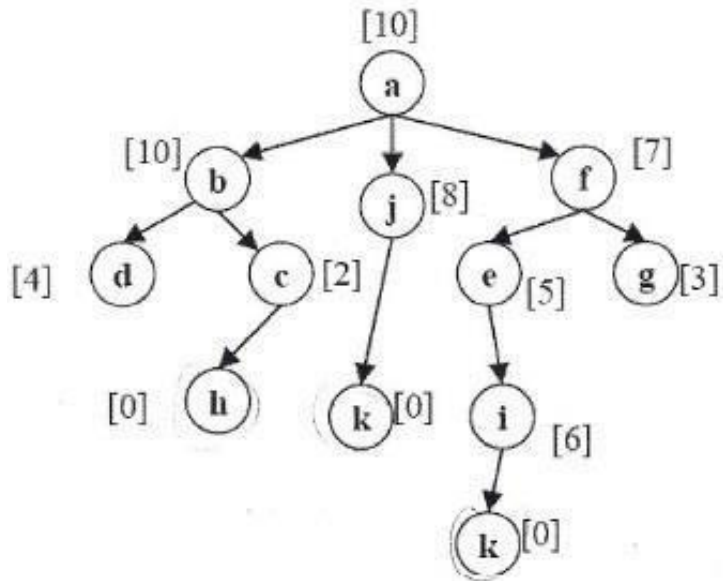
Current

a

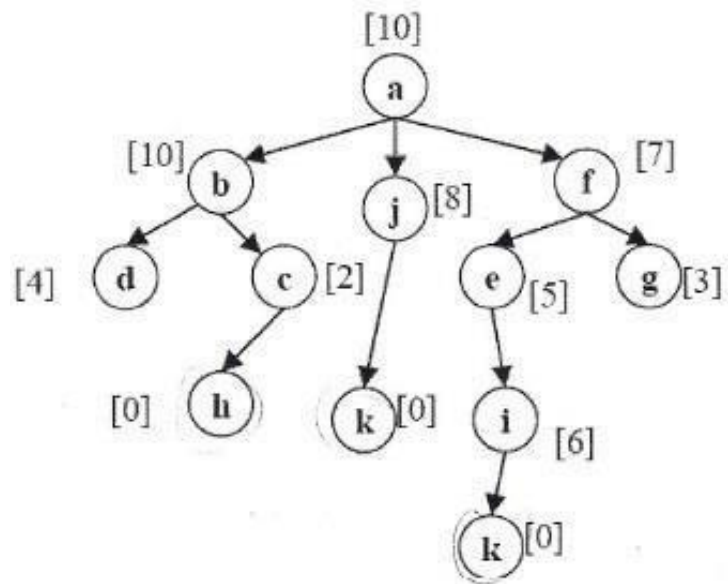
a

Children

*f*7, *j*8, *b*10

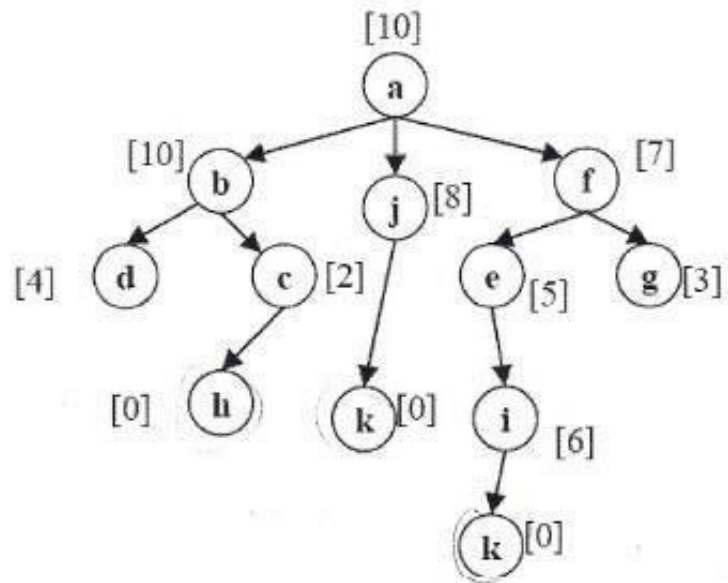


Local Beam Search K=2



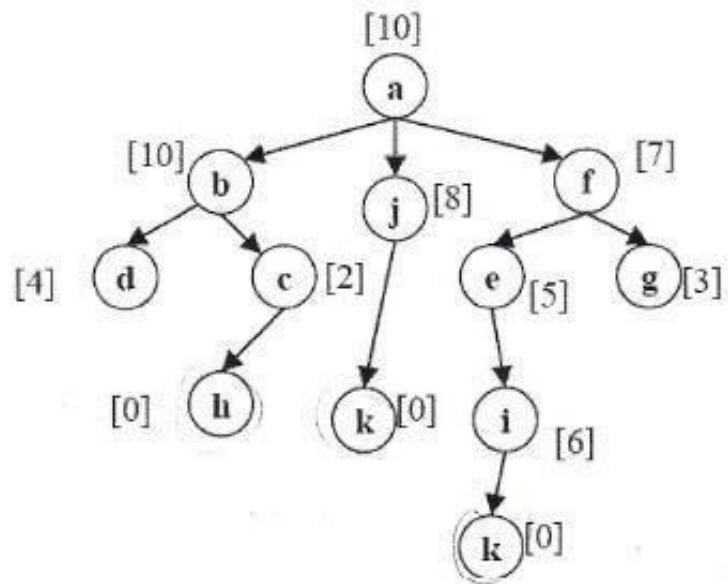
Current		Children
a		---
a		f7, j8, b10
f	j	

Local Beam Search K=2



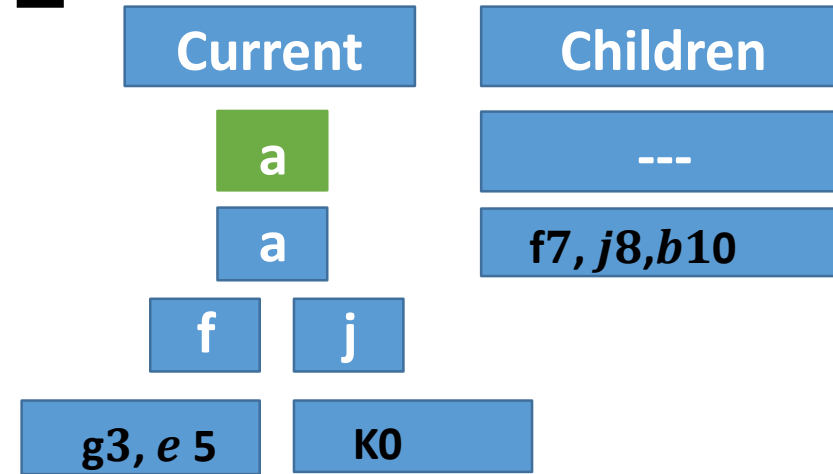
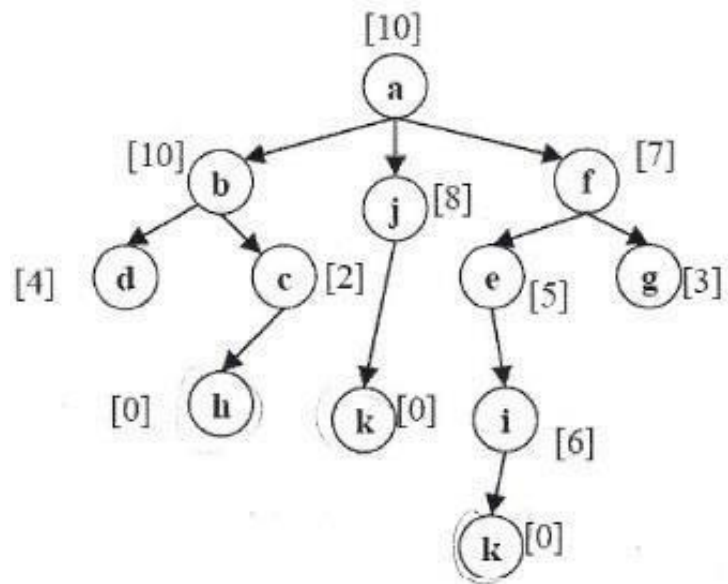
Current	Children
a	---
a	f7, j8, b10
f j	

Local Beam Search K=2

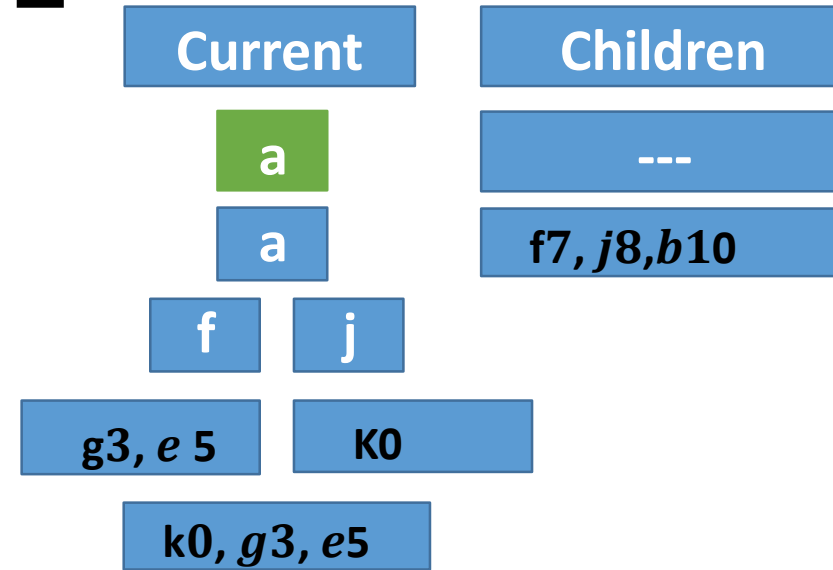
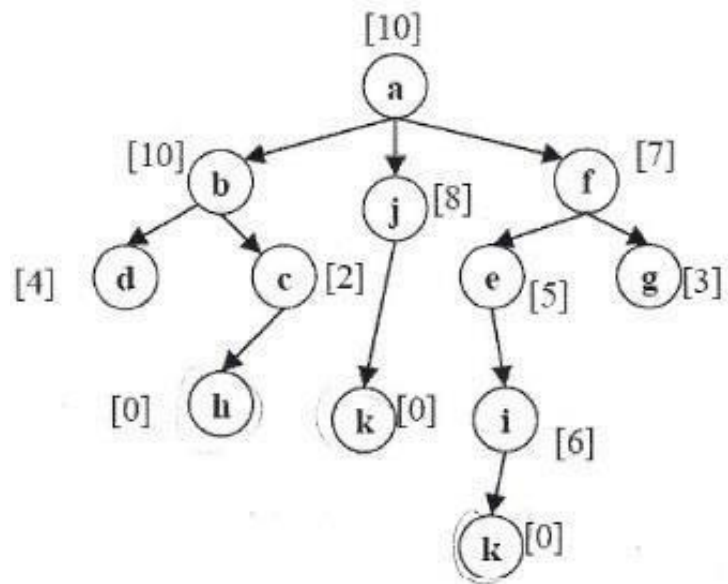


Current	Children
a	---
a	f7, j8, b10
<u>f</u>	
<u>j</u>	

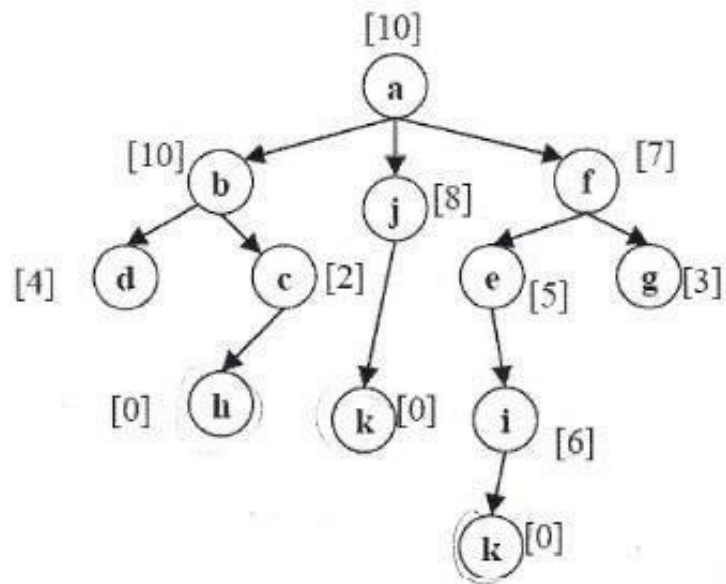
Local Beam Search K=2



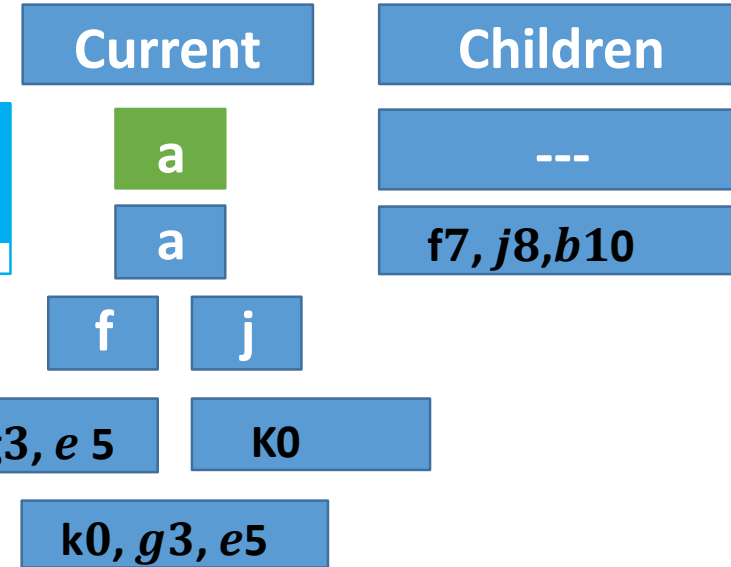
Local Beam Search K=2



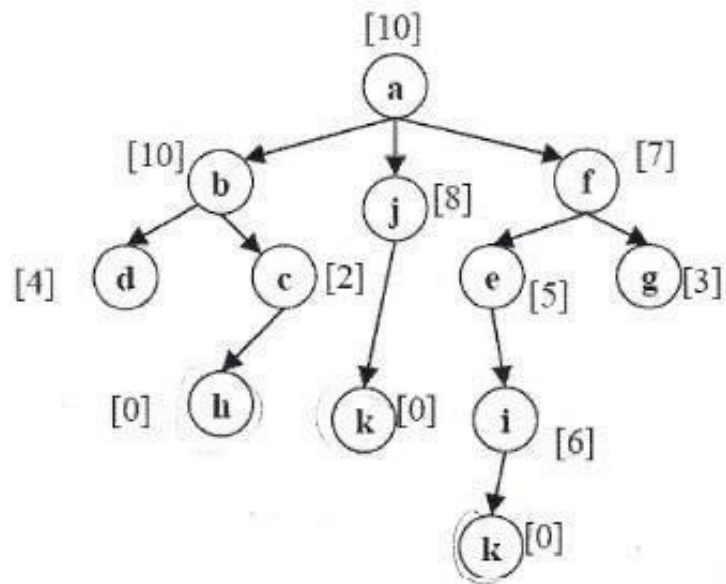
Local Beam Search K=2



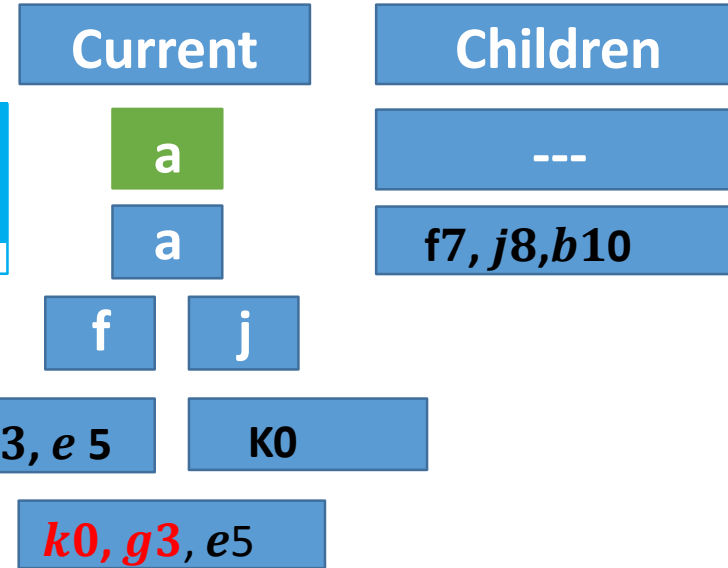
**Best k
Successors**



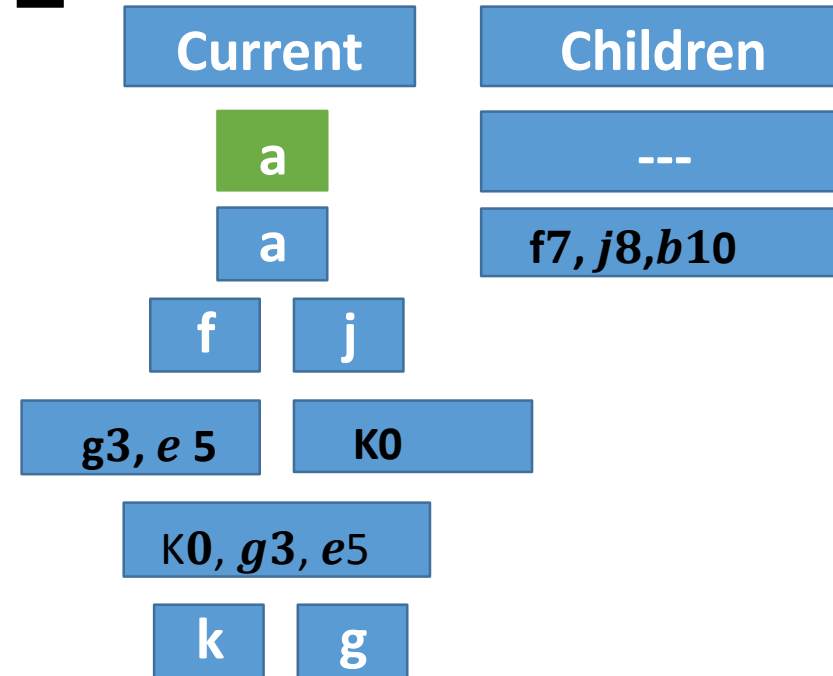
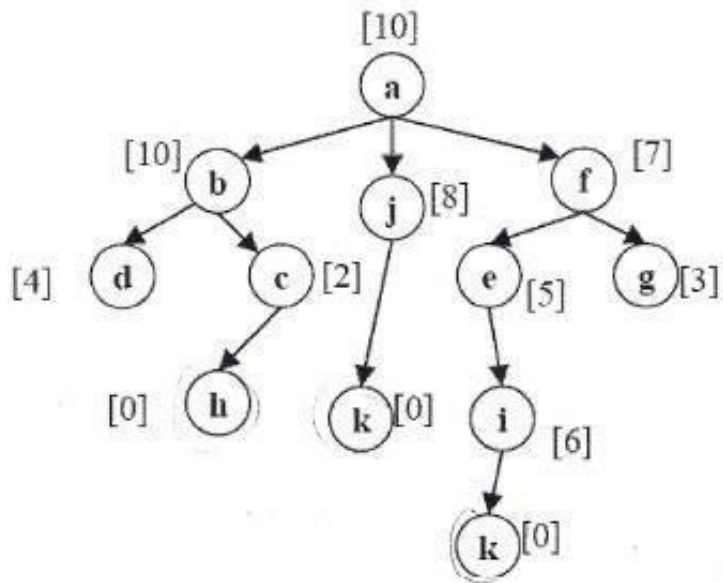
Local Beam Search K=2



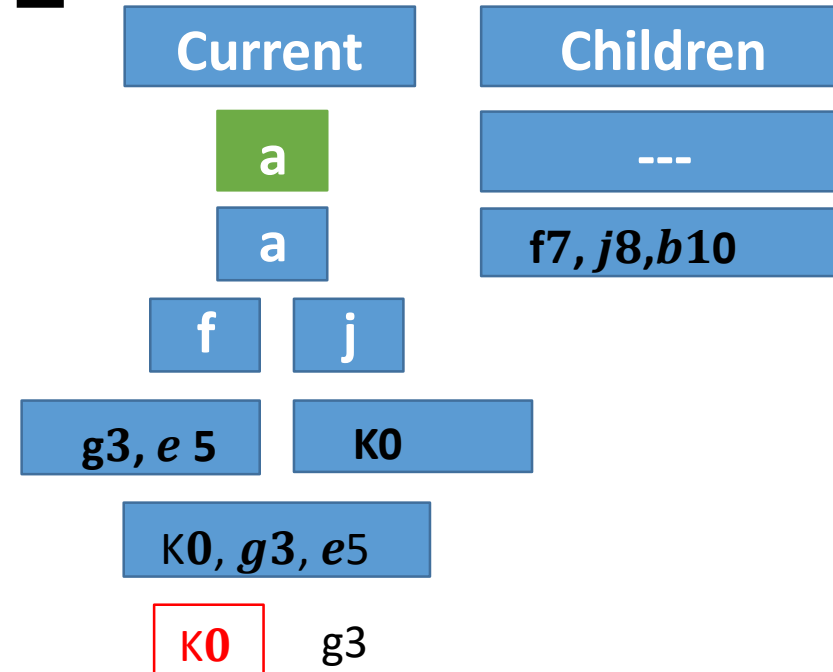
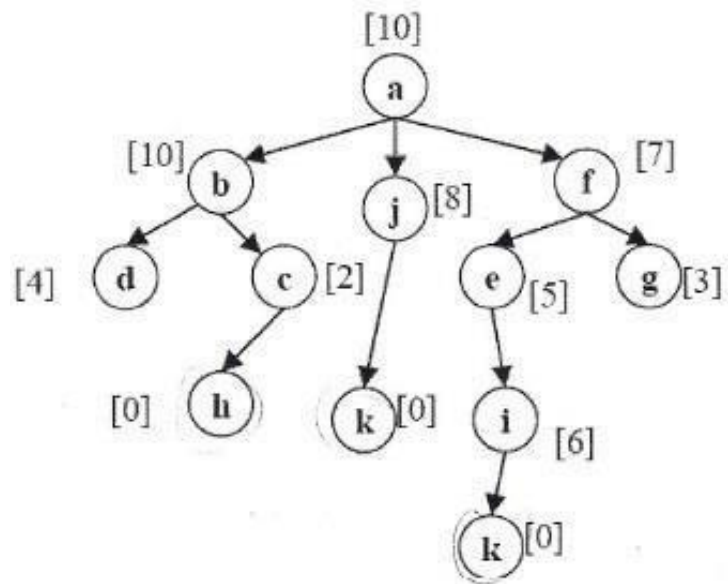
Best k
Successors



Local Beam Search K=2



Local Beam Search K=2



Local Beam Search K=2

